

Adaptive control of multi-joint robotic manipulator systems with RBF neural networks

Yiheng Lyu

Abstract—This paper proposes an adaptive RBF neural control scheme for trajectory tracking of uncertain robotic manipulators. Unknown nonlinear dynamics are approximated online using radial basis function networks within a computed torque control framework. A σ -modified adaptive law is introduced to improve robustness against parameter drift. Lyapunov analysis and simulation results demonstrate the effectiveness of the proposed method.

Index Terms—Robotic manipulators; adaptive control; RBF neural networks; trajectory tracking; computed torque control.

I. INTRODUCTION

ROBOTIC manipulators have become indispensable in modern manufacturing, industrial automation, medical robotics, and human–robot interaction due to their high precision, flexibility, and autonomous operation capabilities. In many robotic applications, accurate trajectory tracking is essential for ensuring motion precision, operational safety, and task efficiency. However, achieving high-performance tracking remains challenging because practical robotic systems are inevitably affected by modeling uncertainties, external disturbances, and unmodeled nonlinear dynamics. These uncertainties may significantly degrade tracking accuracy and even compromise the stability of conventional model-based control methods.

To improve tracking performance under uncertainties, a variety of nonlinear control approaches have been extensively investigated, including computed torque control, robust control, adaptive control, sliding mode control, and intelligent control methods. Among them, computed torque control has been widely adopted because of its systematic design methodology and excellent tracking performance when accurate dynamic models are available. Nevertheless, its effectiveness strongly depends on precise knowledge of the manipulator dynamics, which is often difficult to obtain in practice due to uncertain physical parameters, friction, payload variations, and external disturbances. Adaptive control alleviates part of this limitation by estimating unknown parameters online, while robust control improves disturbance rejection at the expense of increased control effort or conservative gain selection.

In recent years, neural-network-based adaptive control has attracted considerable attention because of its capability to approximate unknown nonlinear functions without requiring explicit mathematical models. In particular, radial basis function (RBF) neural networks have been widely employed owing to their universal approximation capability, simple network

structure, and efficient online learning properties. By integrating RBF neural networks with adaptive control, unknown dynamics can be estimated online and compensated during controller execution, thereby improving tracking accuracy under uncertain environments. Nevertheless, maintaining parameter boundedness and ensuring stable online adaptation remain important issues in adaptive neural control, especially in the presence of approximation errors and persistent disturbances.

Motivated by these observations, this paper investigates the trajectory tracking problem for uncertain robotic manipulators using an adaptive neural control framework based on RBF networks. Unknown nonlinear dynamics are represented as lumped uncertainties and approximated online through RBF neural networks within a computed torque control architecture. Furthermore, a σ -modified adaptive law is incorporated to suppress parameter drift and enhance robustness during online learning. Stability of the overall control system is established through Lyapunov analysis, and numerical simulations are conducted to evaluate the effectiveness of the proposed approach.

The main contributions of this paper are summarized as follows.

An adaptive neural control framework combining computed torque control and RBF neural network approximation is developed for trajectory tracking of uncertain robotic manipulators. A σ -modified adaptive law is incorporated into the online learning process to improve parameter robustness and suppress weight drift. Lyapunov-based stability analysis and numerical simulations demonstrate the effectiveness of the proposed control strategy under model uncertainties and external disturbances.

The remainder of this paper is organized as follows. Section II introduces the preliminary knowledge of robotic manipulator dynamics and RBF neural networks. Section III formulates the trajectory tracking problem. Section IV presents the adaptive controller design together with the corresponding stability analysis. Section V provides numerical simulation results. Finally, Section VI concludes the paper.

II. PRELIMINARIES

A. Radial Basis Function (RBF) Neural Networks

In this paper, Radial Basis Function (RBF) neural networks (NNs) are employed to approximate the unknown nonlinear lumped uncertainty in the robotic manipulator dynamics.

For an n -degree-of-freedom (DOF) robotic system, the input vector of the RBF neural network is selected as the tracking error state:

$$x = X = \begin{bmatrix} e \\ \dot{e} \end{bmatrix} \in \mathbb{R}^{2n}, \quad (1)$$

where $e = q - q_d$ and $\dot{e} = \dot{q} - \dot{q}_d$ denote the position tracking error and velocity tracking error, respectively.

An RBF neural network consists of a nonlinear mapping from the input vector x to the hidden layer and a linear combination of the hidden-layer outputs. The estimated lumped uncertainty is expressed as:

$$\hat{f}(x) = \hat{W}^T h(x), \quad (2)$$

where $\hat{W} \in \mathbb{R}^{m \times n}$ is the estimated weight matrix, m represents the number of hidden neurons, and

$$h(x) = [h_1(x), h_2(x), \dots, h_m(x)]^T \in \mathbb{R}^m \quad (3)$$

denotes the hidden-layer activation vector.

The Gaussian radial basis function of the i -th hidden neuron is defined as:

$$h_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2b_i^2}\right), \quad i = 1, 2, \dots, m, \quad (4)$$

where $c_i \in \mathbb{R}^{2n}$ represents the center vector of the i -th Gaussian basis function and $b_i > 0$ denotes the corresponding width parameter.

According to the universal approximation theorem, for any continuous nonlinear function $f(x)$ defined on a compact set $\Omega_x \subset \mathbb{R}^{2n}$, there exists an ideal constant weight matrix $W^* \in \mathbb{R}^{m \times n}$ such that:

$$f(x) = W^{*T} h(x) + \epsilon, \quad (5)$$

where $\epsilon \in \mathbb{R}^n$ denotes the inherent approximation error of the RBF neural network.

The ideal weight matrix is defined as:

$$W^* \triangleq \arg \min_W \left[\sup_{x \in \Omega_x} \|f(x) - W^T h(x)\| \right]. \quad (6)$$

Furthermore, the approximation error is assumed to be bounded on the compact set Ω_x :

$$\|\epsilon\| \leq \epsilon_N, \quad (7)$$

where $\epsilon_N > 0$ is an unknown positive constant.

Remark 1: The linearly parameterized structure of the RBF neural network indicates that the network output is linear with respect to the adjustable weight matrix $\hat{W}$, while the nonlinear approximation capability is provided by the basis function vector $h(x)$. This property facilitates the design of adaptive weight update laws based on Lyapunov stability analysis.

B. Notation

The dimensions of the main variables used throughout this paper are summarized in Table I.

TABLE I: Dimensions of the Main Variables

Symbol	Dimension
n	Scalar
$q, \dot{q}, \ddot{q}$	$\mathbb{R}^{n \times 1}$
$q_d, \dot{q}_d, \ddot{q}_d$	$\mathbb{R}^{n \times 1}$
$e, \dot{e}$	$\mathbb{R}^{n \times 1}$
$x = X = \begin{bmatrix} e \\ \dot{e} \end{bmatrix}$	$\mathbb{R}^{2n \times 1}$
m	Scalar
$h(x)$	$\mathbb{R}^{m \times 1}$
c_i	$\mathbb{R}^{2n \times 1}$
$C = [c_1, c_2, \dots, c_m]$	$\mathbb{R}^{2n \times m}$
b_i	Scalar
$W^*, \hat{W}, \tilde{W}$	$\mathbb{R}^{m \times n}$
$f(x), \hat{f}(x)$	$\mathbb{R}^{n \times 1}$
ϵ	$\mathbb{R}^{n \times 1}$
P	$\mathbb{R}^{2n \times 2n}$
$B = \begin{bmatrix} 0 \\ M_0^{-1}(q) \end{bmatrix}$	$\mathbb{R}^{2n \times n}$
$d(t)$	$\mathbb{R}^{n \times 1}$

III. PROBLEM FORMULATION

A. System Dynamics

Consider an n -degree-of-freedom (DOF) rigid robotic manipulator governed by the Euler-Lagrange equation

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau + d, \quad (8)$$

where $q, \dot{q}, \ddot{q} \in \mathbb{R}^n$ denote the joint position, velocity, and acceleration vectors, respectively. The matrix $M(q) \in \mathbb{R}^{n \times n}$ denotes the inertia matrix, $C(q, \dot{q}) \in \mathbb{R}^{n \times n}$ is the Coriolis and centrifugal matrix, $G(q) \in \mathbb{R}^n$ is the gravity vector, $\tau \in \mathbb{R}^n$ is the control input, and $d \in \mathbb{R}^n$ represents the lumped external disturbance and unmodeled dynamics.

The following standard properties of Euler-Lagrange systems are adopted throughout this paper.

Assumption 1: The unknown external disturbance $d(t)$ is piecewise continuous and uniformly bounded, i.e., there exists an unknown positive constant d_m such that $\|d(t)\| \leq d_m, \forall t \geq 0$. Furthermore, if the disturbance depends on the system states $d(x, t)$, it is assumed to be Lipschitz continuous with respect to x over a compact domain Ω_x , ensuring the existence of a bounded RBF approximation error.

Property 1: The inertia matrix $M(q)$ is symmetric and uniformly positive definite. Specifically, there exist two positive constants m_1 and m_2 satisfying

$$m_1 I \leq M(q) \leq m_2 I, \quad (9)$$

for all $q \in \mathbb{R}^n$.

Property 2: The matrix $\dot{M}(q) - 2C(q, \dot{q})$ is skew-symmetric, i.e.,

$$x^T (\dot{M}(q) - 2C(q, \dot{q})) x = 0, \quad (10)$$

for any $x \in \mathbb{R}^n$.

In practical robotic systems, the exact dynamic model is unavailable due to parameter uncertainties, payload variations, friction, and unmodeled dynamics. Accordingly, the manipulator dynamics are decomposed into a nominal model and an uncertainty term as

$$M(q) = M_0(q) + \Delta M(q), \quad (11)$$

$$C(q, \dot{q}) = C_0(q, \dot{q}) + \Delta C(q, \dot{q}), \quad (12)$$

$$G(q) = G_0(q) + \Delta G(q), \quad (13)$$

where $M_0(q)$, $C_0(q, \dot{q})$, and $G_0(q)$ denote the nominal inertia, Coriolis/centrifugal, and gravity terms, respectively.

Substituting the above decompositions into (8) yields

$$(M_0 + \Delta M) \ddot{q} + (C_0 + \Delta C) \dot{q} + (G_0 + \Delta G) = \tau + d. \quad (14)$$

Rearranging the above equation gives

$$M_0(q) \ddot{q} + C_0(q, \dot{q}) \dot{q} + G_0(q) = \tau + f(q, \dot{q}, \ddot{q}), \quad (15)$$

where

$$f(q, \dot{q}, \ddot{q}) = d - \Delta M(q) \ddot{q} - \Delta C(q, \dot{q}) \dot{q} - \Delta G(q). \quad (16)$$

The nonlinear function $f(\cdot)$ represents the lumped uncertainty consisting of modeling errors, parameter variations, external disturbances, and unmodeled dynamics.

B. Tracking Control Problem

Assume that the desired joint trajectory satisfies

$$q_d, \dot{q}_d, \ddot{q}_d \in \mathcal{L}_\infty. \quad (17)$$

Define the tracking error and its derivative as

$$e = q - q_d, \quad \dot{e} = \dot{q} - \dot{q}_d. \quad (18)$$

Differentiating (18) yields

$$\ddot{e} = \ddot{q} - \ddot{q}_d. \quad (19)$$

The desired tracking performance is specified by the second-order error dynamics

$$\ddot{e} + K_v \dot{e} + K_p e = 0, \quad (20)$$

where $K_p, K_v \in \mathbb{R}^{n \times n}$ are symmetric positive-definite gain matrices.

Assumption 2: The external disturbance is bounded, i.e.,

$$\|d(t)\| \leq d_{\max}, \quad (21)$$

where $d_{\max}$ is an unknown positive constant.

Substituting (19) into (20) yields the desired joint acceleration

$$\ddot{q} = \ddot{q}_d - K_v \dot{e} - K_p e. \quad (22)$$

The control objective is to design an adaptive neural controller such that all closed-loop signals remain bounded and the tracking error converges to a sufficiently small neighborhood of the origin despite the presence of lumped uncertainties.

IV. CONTROLLER DESIGN

A. Nominal Controller Design

To achieve the desired tracking objective, define the auxiliary control input as

$$v = \ddot{q}_d - K_v \dot{e} - K_p e, \quad (23)$$

where $K_p, K_v \in \mathbb{R}^{n \times n}$ are symmetric positive-definite gain matrices.

Based on the nominal manipulator dynamics, the control law is designed as

$$\tau = M_0(q)v + C_0(q, \dot{q})\dot{q} + G_0(q) - \hat{f}(x), \quad (24)$$

where $\hat{f}(x)$ denotes the neural-network estimation of the lumped uncertainty.

Remark 2: The introduction of the virtual control input v decouples the controller into two functional modules. The outer-loop controller computes the virtual acceleration command v to achieve the desired tracking performance, whereas the inner-loop controller realizes this command by generating the actual joint torque τ through dynamic inversion and RBF neural network compensation. This modular design separates trajectory tracking from dynamic compensation, thereby simplifying controller design and implementation.

Substituting (24) into the nominal dynamics (15) yields

$$M_0(q) \ddot{q} = M_0(q)v + f(q, \dot{q}, \ddot{q}) - \hat{f}(x). \quad (25)$$

Using the definition of v in (23), one obtains

$$M_0(q) (\ddot{q} - \ddot{q}_d + K_v \dot{e} + K_p e) = f - \hat{f}(x). \quad (26)$$

Since

$$\ddot{e} = \ddot{q} - \ddot{q}_d, \quad (27)$$

the closed-loop tracking error dynamics can be written as

$$\ddot{e} + K_v \dot{e} + K_p e = M_0^{-1}(q) (f - \hat{f}(x)). \quad (28)$$

B. State-Space Representation

Define the state vector as

$$X = \begin{bmatrix} e \\ \dot{e} \end{bmatrix}, \quad \dot{X} = \begin{bmatrix} \dot{e} \\ \ddot{e} \end{bmatrix}. \quad (29)$$

Substituting (28) into (29) gives

$$\dot{X} = AX + B (f - \hat{f}(x)), \quad (30)$$

where

$$A = \begin{bmatrix} 0 & I \\ -K_p & -K_v \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ M_0^{-1}(q) \end{bmatrix}. \quad (31)$$

Equation (30) reformulates the second-order closed-loop system as a first-order state-space representation, facilitating the subsequent stability analysis and adaptive controller design.

C. RBF Neural Network Approximation

To compensate for the unknown lumped uncertainty $f(x)$, an RBF neural network is employed.

According to the universal approximation theorem, there exists an ideal constant weight matrix W^* such that

$$f(x) = W^{*T}h(x) + \varepsilon, \quad (32)$$

where $h(x) \in \mathbb{R}^m$ denotes the Gaussian basis function vector, $W^* \in \mathbb{R}^{m \times n}$ is the ideal weight matrix, and $\varepsilon \in \mathbb{R}^n$ represents the bounded approximation error satisfying

$$\|\varepsilon\| \leq \bar{\varepsilon}. \quad (33)$$

The neural-network output is given by

$$\hat{f}(x) = \hat{W}^T h(x), \quad (34)$$

where $\hat{W}$ denotes the online estimate of the ideal weight matrix.

Define the weight estimation error as

$$\tilde{W} = W^* - \hat{W}. \quad (35)$$

Substituting (32) and (34) into (30) yields

$$\dot{X} = AX + B \left(\tilde{W}^T h(x) + \varepsilon \right), \quad (36)$$

which will be employed in the subsequent Lyapunov stability analysis.

V. STABILITY ANALYSIS

In this section, the stability of the proposed adaptive neural controller is established using the Lyapunov direct method. For simplicity of the theoretical analysis, the nominal closed-loop system matrix A defined in (31) is assumed to be Hurwitz. Consequently, for any given symmetric positive-definite matrix $Q \in \mathbb{R}^{2n \times 2n}$, there exists a unique symmetric positive-definite matrix $P \in \mathbb{R}^{2n \times 2n}$ satisfying the Lyapunov equation

$$A^T P + P A = -Q, \quad (37)$$

where $Q = Q^T > 0$ and $P = P^T > 0$.

A. Lyapunov Stability Analysis

Consider the following composite Lyapunov candidate function

$$V = \frac{1}{2} X^T P X + \frac{1}{2\gamma} \text{tr} \left(\tilde{W}^T \tilde{W} \right), \quad (38)$$

where $\gamma > 0$ denotes the adaptation gain.

Differentiating (38) with respect to time yields

$$\dot{V} = \frac{1}{2} \dot{X}^T P X + \frac{1}{2} X^T P \dot{X} + \frac{1}{\gamma} \text{tr} \left(\dot{\tilde{W}}^T \tilde{W} \right). \quad (39)$$

Since the ideal weight matrix W^* is constant, one has

$$\dot{\tilde{W}} = -\dot{\hat{W}}. \quad (40)$$

Substituting the closed-loop system (36)

$$\dot{X} = AX + B \left(\tilde{W}^T h(x) + \varepsilon \right) \quad (41)$$

into (39) gives

$$\begin{aligned} \dot{V} = & \frac{1}{2} \left[AX + B \left(\tilde{W}^T h + \varepsilon \right) \right]^T P X \\ & + \frac{1}{2} X^T P \left[AX + B \left(\tilde{W}^T h + \varepsilon \right) \right] \\ & - \frac{1}{\gamma} \text{tr} \left(\dot{\tilde{W}}^T \tilde{W} \right). \end{aligned} \quad (42)$$

Expanding (42) yields

$$\begin{aligned} \dot{V} = & \frac{1}{2} X^T (A^T P + P A) X \\ & + X^T P B \tilde{W}^T h + X^T P B \varepsilon \\ & - \frac{1}{\gamma} \text{tr} \left(\dot{\tilde{W}}^T \tilde{W} \right). \end{aligned} \quad (43)$$

Using the Lyapunov equation (37) $A^T P + P A = -Q$, one obtains

$$\dot{V} = -\frac{1}{2} X^T Q X + X^T P B \tilde{W}^T h + X^T P B \varepsilon - \frac{1}{\gamma} \text{tr} \left(\dot{\tilde{W}}^T \tilde{W} \right). \quad (44)$$

By exploiting the cyclic property of the trace operator,

$$X^T P B \tilde{W}^T h = \text{tr} \left(\tilde{W}^T h X^T P B \right), \quad (45)$$

the derivative of the Lyapunov function can be rewritten as

$$\begin{aligned} \dot{V} = & -\frac{1}{2} X^T Q X + X^T P B \varepsilon \\ & + \frac{1}{\gamma} \text{tr} \left[\tilde{W}^T \left(\gamma h X^T P B - \dot{\tilde{W}} \right) \right]. \end{aligned} \quad (46)$$

B. Adaptive Law with σ -Modification

To eliminate the neural-network parameter estimation term in (44), the following trace identity is first employed:

$$X^T P B \tilde{W}^T h(x) = \text{tr} \left[\tilde{W}^T h(x) X^T P B \right]. \quad (47)$$

Substituting (47) into (44) yields

$$\dot{V} = -\frac{1}{2} X^T Q X + X^T P B \varepsilon + \frac{1}{\gamma} \text{tr} \left[\tilde{W}^T \left(\gamma h(X) X^T P B - \dot{\tilde{W}} \right) \right]. \quad (48)$$

Accordingly, the adaptive law is selected as

$$\dot{\tilde{W}} = \gamma h(X) X^T P B, \quad (49)$$

which completely cancels the parameter estimation term in the Lyapunov derivative. Consequently,

$$\dot{V} = -\frac{1}{2} X^T Q X + X^T P B \varepsilon. \quad (50)$$

Since the neural-network approximation error is bounded, namely,

$$\|\varepsilon\| \leq \varepsilon_0, \quad (51)$$

the Lyapunov derivative satisfies

$$\dot{V} \leq -\frac{1}{2}\lambda_{\min}(Q)\|X\|^2 + \|PB\|\varepsilon_0\|X\|. \quad (52)$$

Therefore, whenever

$$\|X\| \geq \frac{2\|PB\|\varepsilon_0}{\lambda_{\min}(Q)}, \quad (53)$$

one has $\dot{V} \leq 0$. Hence, the closed-loop system is uniformly ultimately bounded.

Remark 3: The adaptive law (49) is a pure integrator driven by $h(X)X^T PB$, which depends only on the state X and contains no term that penalizes the magnitude of $\hat{W}$ itself. Consequently, (49) alone cannot guarantee the boundedness of $\hat{W}$: whenever a persistent disturbance or a nonvanishing approximation error ε is present, the tracking error X no longer converges to the origin but only to a bounded neighborhood as shown in (53), so the driving term $h(X)X^T PB$ generally never vanishes. If this residual excitation does not average out to zero, $\hat{W}$ may keep accumulating in a fixed direction indefinitely, leading to unbounded parameter estimates even though the tracking error itself remains bounded. This is the well-known parameter drift phenomenon in adaptive control.

To suppress parameter drift and endow the adaptive law with an explicit self-decaying mechanism against persistent disturbances and approximation errors, the following σ -modification is introduced:

$$\dot{\hat{W}} = \gamma h(X)X^T PB - \sigma\gamma\|X\|\hat{W}, \quad (54)$$

where $\sigma > 0$ denotes the leakage coefficient.

Substituting (54) into (48) gives

$$\dot{V} = -\frac{1}{2}X^T QX + X^T PB\varepsilon + \sigma\|X\| \operatorname{tr}(\tilde{W}^T \hat{W}). \quad (55)$$

C. Uniform Ultimate Boundedness Analysis

To analyze the influence of the σ -modification term, the following identity is first established:

$$\operatorname{tr}(\tilde{W}^T \hat{W}) = \operatorname{tr}(\tilde{W}^T W^*) - \operatorname{tr}(\tilde{W}^T \tilde{W}). \quad (56)$$

Applying the Cauchy–Schwarz inequality gives

$$\operatorname{tr}(\tilde{W}^T W^*) \leq \|\tilde{W}\|_F \|W^*\|_F \leq \|\tilde{W}\|_F W_{\max}, \quad (57)$$

where $W_{\max}$ is a positive constant satisfying

$$\|W^*\|_F \leq W_{\max}.$$

Substituting (56) and (57) into (55) yields

$$\begin{aligned} \dot{V} \leq & -\frac{1}{2}\lambda_{\min}(Q)\|X\|^2 + \|PB\|\varepsilon_0\|X\| \\ & + \sigma\|X\|\|\tilde{W}\|_F W_{\max} - \sigma\|X\|\|\tilde{W}\|_F^2. \end{aligned} \quad (58)$$

TABLE II: Definitions of Control Modes and Adaptive Laws

Parameter	Value	Description
S_1	1	First adaptive law
S_1	2	Second adaptive law with σ -modification
S	1	Nominal model-based controller without compensation
S	2	Controller with exact uncertainty compensation
S	3	Controller with RBF neural network compensation

Factoring out $\|X\|$ gives

$$\begin{aligned} \dot{V} \leq & -\|X\| \left[\frac{1}{2}\lambda_{\min}(Q)\|X\| + \sigma\|\tilde{W}\|_F^2 \right. \\ & \left. - \sigma W_{\max}\|\tilde{W}\|_F - \|PB\|\varepsilon_0 \right]. \end{aligned} \quad (59)$$

Completing the square gives

$$\begin{aligned} \sigma\|\tilde{W}\|_F^2 - \sigma W_{\max}\|\tilde{W}\|_F = & \sigma \left(\|\tilde{W}\|_F - \frac{1}{2}W_{\max} \right)^2 \\ & - \frac{\sigma}{4}W_{\max}^2. \end{aligned} \quad (60)$$

Substituting (60) into (59) yields

$$\begin{aligned} \dot{V} \leq & -\|X\| \left[\frac{1}{2}\lambda_{\min}(Q)\|X\| + \sigma \left(\|\tilde{W}\|_F - \frac{1}{2}W_{\max} \right)^2 \right. \\ & \left. - \left(\frac{\sigma}{4}W_{\max}^2 + \|PB\|\varepsilon_0 \right) \right]. \end{aligned} \quad (61)$$

Therefore,

$$\|X\| > \frac{2}{\lambda_{\min}(Q)} \left(\frac{\sigma}{4}W_{\max}^2 + \|PB\|\varepsilon_0 \right), \quad (62)$$

and

$$\|\tilde{W}\|_F > \frac{1}{2}W_{\max} + \sqrt{\frac{1}{4}W_{\max}^2 + \frac{\|PB\|\varepsilon_0}{\sigma}}, \quad (63)$$

imply that

$$\dot{V} < 0.$$

Hence, both the system state X and the neural-network weight estimation error $\tilde{W}$ are uniformly ultimately bounded.

Remark 4: The conditions in (62) and (63) are alternative conditions. The satisfaction of either condition is sufficient to guarantee that $\dot{V} < 0$, which ensures the uniform ultimate boundedness of the closed-loop system.

Consequently, all closed-loop signals remain bounded, and the tracking error converges to a compact neighborhood of the origin despite the presence of modeling uncertainties and external disturbances.

VI. SIMULATION

Numerical simulations are conducted to verify the effectiveness of the proposed RBF neural network-based adaptive control strategy. Two nonlinear systems with different uncertainty characteristics are considered. The first example investigates the approximation capability of the RBF compensator under non-smooth disturbances, while the second example evaluates the proposed controller for a multi-joint robotic manipulator with dynamic coupling and lumped uncertainties.

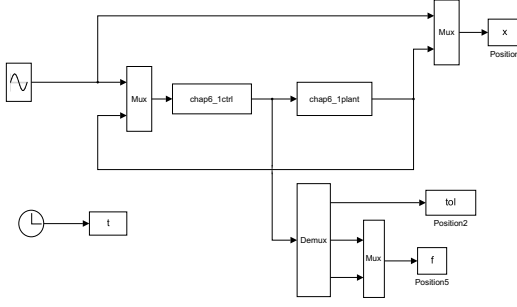


Fig. 1: Simulink implementation of the nonlinear servo system and the proposed control framework.

A. Example 1: Nonlinear Servo System with Non-Smooth Disturbance

Consider a single-degree-of-freedom nonlinear servo system described by

$$M\ddot{q} = \tau + d(\dot{q}), \quad (64)$$

where $M = 10$ and the disturbance term is given by

$$d(\dot{q}) = -15\dot{q} - 30 \operatorname{sgn}(\dot{q}). \quad (65)$$

Before presenting the simulation results, the overall simulation configuration is introduced. The Simulink implementation of the proposed control framework is illustrated in Fig. 1. The simulation platform consists of the nonlinear servo system, the controller module, the reference trajectory generator, and the uncertainty compensation module.

The controller receives the desired trajectory and the measured system states, and generates the control input τ . Depending on the selected control mode, the uncertainty compensation term is either disabled ($S = 1$), obtained from the known disturbance model ($S = 2$), or generated by the RBF neural network ($S = 3$).

The disturbance consists of viscous friction and Coulomb friction components, which introduces a non-smooth uncertainty into the closed-loop system.

The desired trajectory is selected as

$$q_d(t) = \sin(t) \text{ rad}, \quad (66)$$

with the initial condition

$$[q(0), \dot{q}(0)]^T = [0.6, 0]^T.$$

The controller parameters are selected as

$$Q = 50I_2, \quad \gamma = 200, \quad \sigma = 0.001.$$

The RBF neural network employs Gaussian basis functions with different numbers of hidden nodes. Two configurations with $m = 5$ and $m = 19$ are considered.

For the case of $m = 5$, the centers are selected as

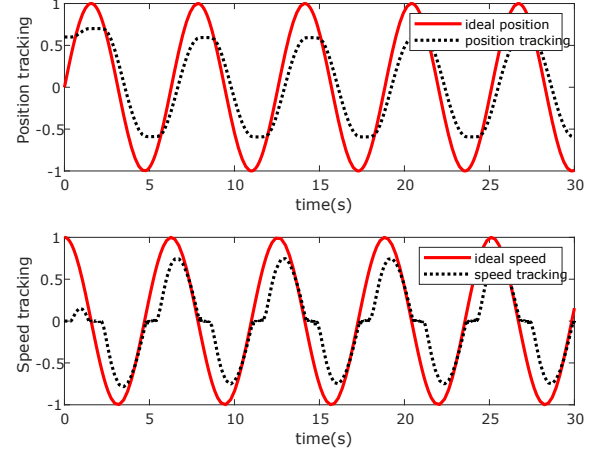


Fig. 2: Tracking performance of the nominal controller ($S = 1$).

$$c_i = [\alpha_i, \alpha_i]^T,$$

where

$$\alpha_i \in \{-1, -0.5, 0, 0.5, 1\}.$$

For the case of $m = 19$, a denser distribution of centers is adopted:

$$\alpha_i \in \{-1, -8/9, \dots, 0, \dots, 8/9, 1\}.$$

The Gaussian basis functions are defined as

$$h_i(\xi) = \exp\left(-\frac{\|\xi - c_i\|^2}{2b^2}\right),$$

where the width parameter is selected as $b = 5$ for all basis functions. The initial neural network weights are set as $\hat{W}(0) = 0$.

To evaluate the effectiveness of the proposed adaptive compensation strategy, three different compensation modes are considered:

- $S = 1$: nominal model-based controller without uncertainty compensation;
- $S = 2$: exact compensation controller assuming that the uncertainty $d(\dot{q})$ is completely known;
- $S = 3$: RBF neural network compensation controller, where the unknown uncertainty is approximated online.

For $S = 3$, two adaptive laws are investigated: $S_1 = 1$ and $S_1 = 2$.

1) *Nominal Model-Based Controller ($S = 1$):* In this case, only the nominal model is used in the controller design. The disturbance term is not compensated, and the adaptive law selection parameter S_1 has no influence because the RBF network is not involved.

The tracking response is shown in Fig. 2, while the uncertainty response is presented in Fig. 3.

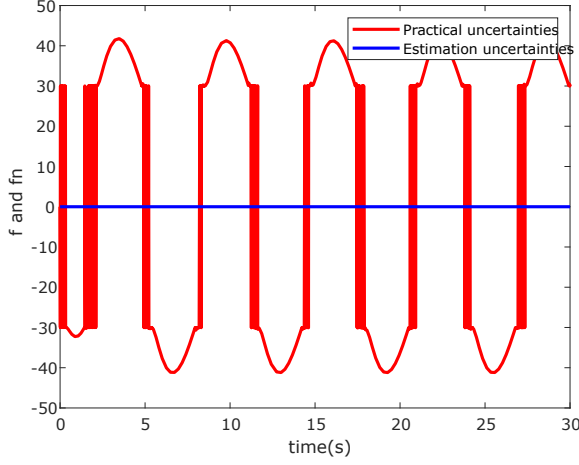


Fig. 3: Unknown disturbance and estimation result under the nominal controller ($S = 1$).

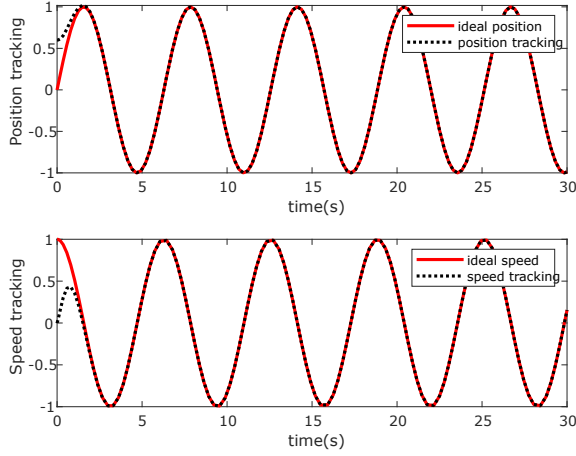


Fig. 4: Tracking performance with exact uncertainty compensation ($S = 2$).

As observed in Fig. 2, the nominal controller cannot completely reject the non-smooth friction disturbance, resulting in a non-zero tracking error.

Since no compensation mechanism is activated, the estimated uncertainty remains zero, as shown in Fig. 3.

2) *Exact Compensation Controller ($S = 2$)*: For $S = 2$, the uncertainty term is assumed to be known exactly. Therefore, the compensation input directly cancels the disturbance:

$$\tau_c = -d(\dot{q}).$$

The adaptive law parameter S_1 does not affect the control performance because the RBF neural network is not used.

As shown in Fig. 4, the tracking error converges rapidly because the disturbance is completely compensated.

Different from the RBF compensation case, the uncertainty estimation is not generated by the neural network. Instead, the exact disturbance model is directly used for compensation.

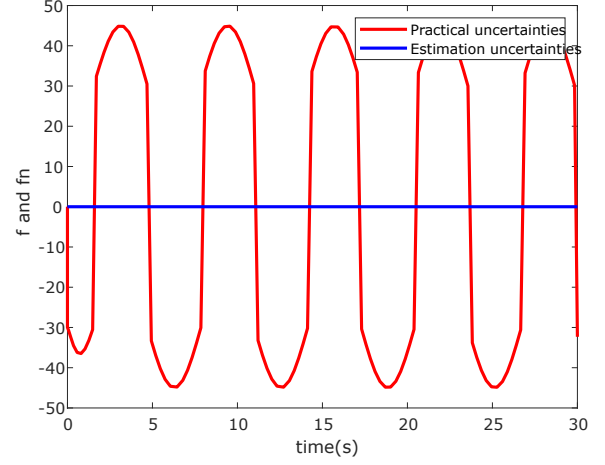


Fig. 5: Exact disturbance compensation result ($S = 2$).

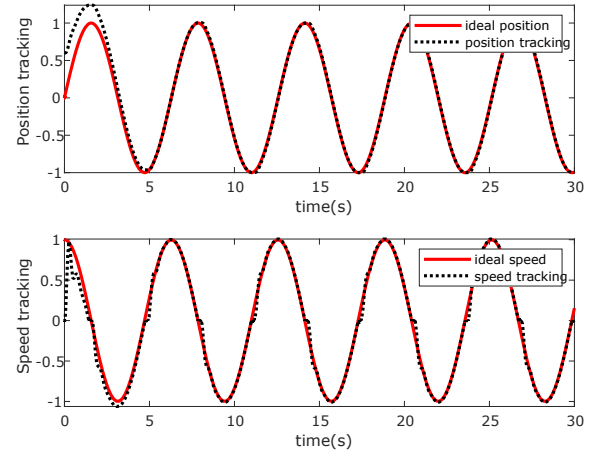


Fig. 6: Tracking performance of the RBF adaptive controller with the first adaptive law ($S = 3, S_1 = 1$).

3) *RBF Neural Network Compensation ($S = 3$)*: When $S = 3$, the explicit expression of the nonlinear disturbance is assumed to be unknown to the controller. Therefore, an RBF neural network is introduced to approximate the lumped uncertainty:

$$\hat{f}(x) = \hat{W}^T h(\xi), \quad (67)$$

where $\hat{W}$ denotes the adaptive weight vector and $h(\xi)$ represents the Gaussian basis function vector.

Unlike the previous two cases, the disturbance compensation is generated by the neural network rather than directly using the known disturbance model. Therefore, the adaptive law plays an important role in determining the convergence behavior of the estimated uncertainty.

a) *First adaptive law ($S = 3, S_1 = 1$)*: Firstly, the RBF controller with the first adaptive law is investigated. The tracking performance and uncertainty approximation results are shown in Fig. 6 and Fig. 7, respectively.

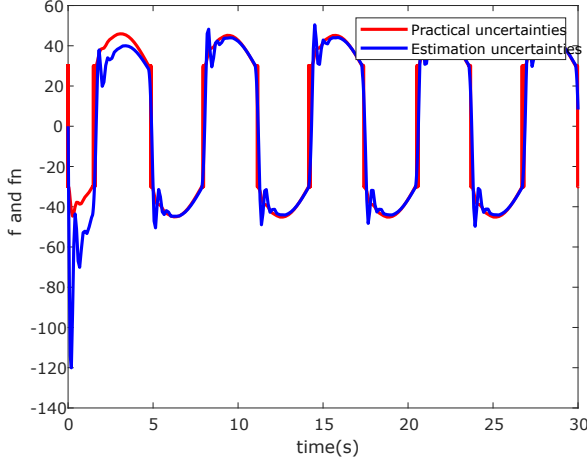


Fig. 7: Approximation of the nonlinear disturbance using the RBF neural network with the first adaptive law ($S = 3, S_1 = 1$).

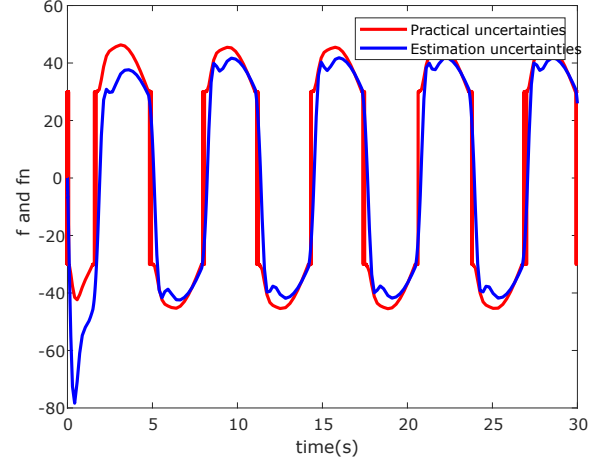


Fig. 9: RBF approximation result with five hidden nodes ($S = 3, S_1 = 2, m = 5$).

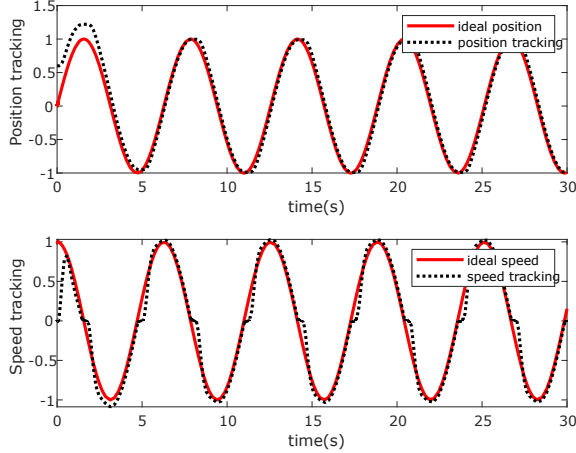


Fig. 8: Tracking response using the second adaptive law with five hidden nodes ($S = 3, S_1 = 2, m = 5$).

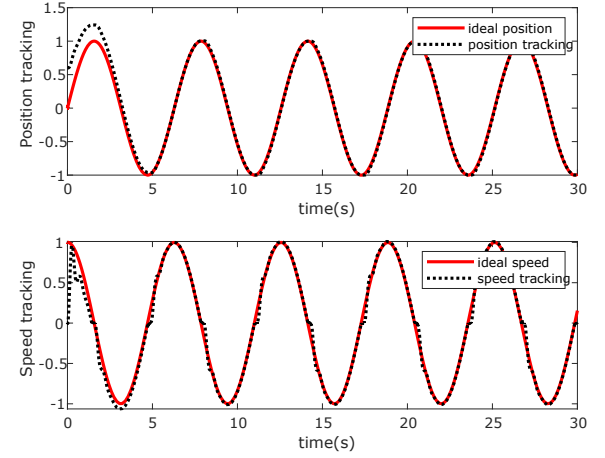


Fig. 10: Tracking response using the second adaptive law with nineteen hidden nodes ($S = 3, S_1 = 2, m = 19$).

As shown in Fig. 6, the RBF compensation effectively reduces the tracking error compared with the nominal controller. The approximation result in Fig. 7 demonstrates that the neural network is able to learn the unknown disturbance online.

b) Second adaptive law with $m = 5$ hidden nodes ($S = 3, S_1 = 2$): Next, the second adaptive law is evaluated with an RBF neural network containing five hidden nodes.

The corresponding tracking response and disturbance approximation result are shown in Fig. 8 and Fig. 9.

With a limited number of hidden nodes, the RBF network provides a coarse approximation of the nonlinear disturbance. The residual approximation error leads to a relatively larger tracking deviation.

c) Second adaptive law with $m = 19$ hidden nodes ($S = 3, S_1 = 2$): To further investigate the influence of the approximation capability, the number of hidden nodes is increased to $m = 19$.

The corresponding results are presented in Fig. 10 and

Fig. 11.

Compared with the case of $m = 5$, the denser distribution of Gaussian basis functions improves the approximation capability of the RBF neural network. Consequently, the estimated uncertainty is closer to the actual disturbance, which leads to improved tracking performance.

d) Summary: The simulation results verify the effectiveness of the RBF-based adaptive compensation strategy.

When the uncertainty model is unavailable, the RBF neural network can successfully approximate the unknown nonlinear disturbance and reduce the tracking error.

Furthermore, increasing the hidden-layer size from $m = 5$ to $m = 19$ enhances the approximation capability of the neural network. The comparison also shows that the adaptive law selection affects the weight update behavior and uncertainty estimation performance.

4) Discussion: The simulation results verify the effectiveness of different compensation strategies.

The nominal controller ($S = 1$) suffers from residual

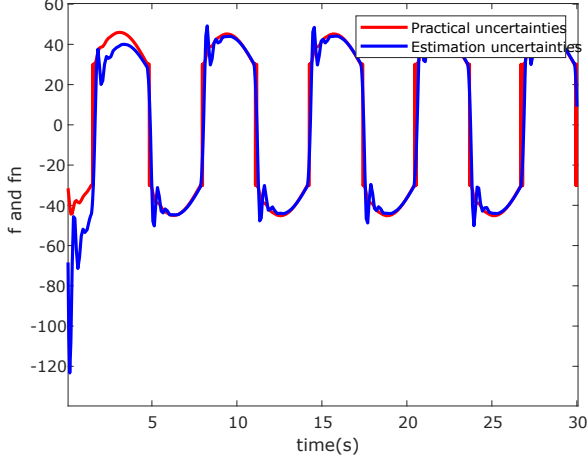


Fig. 11: RBF approximation result with nineteen hidden nodes ($S = 3, S_1 = 2, m = 19$).

tracking errors because the disturbance is completely uncompensated.

The exact compensation controller ($S = 2$) achieves the best tracking performance since the disturbance is assumed to be completely known.

For the practical case where the uncertainty model is unavailable, the RBF-based adaptive controller ($S = 3$) successfully approximates the unknown disturbance and achieves improved tracking performance.

Moreover, the comparison between $S_1 = 1$ and $S_1 = 2$ demonstrates that the second adaptive law provides improved weight regulation while preserving tracking accuracy.

B. Example 2: Two-Link Robotic Manipulator with Coupled Uncertainties

A two-link planar robotic manipulator is considered to further evaluate the proposed adaptive control strategy in the presence of nonlinear dynamic coupling, parameter uncertainties, and external disturbances. The simulation configuration implemented in Simulink is shown in Fig. 12.

The nominal dynamics of the robotic manipulator are described as

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau + d, \quad (68)$$

where the inertia matrix, Coriolis matrix, and gravity vector are given by

$$M(q) = \begin{bmatrix} v + q_{01} + 2q_{02} \cos(q_2) & q_{01} + q_{02} \cos(q_2) \\ q_{01} + q_{02} \cos(q_2) & q_{01} \end{bmatrix}, \quad (69)$$

$$C(q, \dot{q}) = \begin{bmatrix} -q_{02}\dot{q}_2 \sin(q_2) & -q_{02}(\dot{q}_1 + \dot{q}_2) \sin(q_2) \\ q_{02}\dot{q}_1 \sin(q_2) & 0 \end{bmatrix}, \quad (70)$$

$$G(q) = \begin{bmatrix} 15g \cos(q_1) + 8.75g \cos(q_1 + q_2) \\ 8.75g \cos(q_1 + q_2) \end{bmatrix}. \quad (71)$$

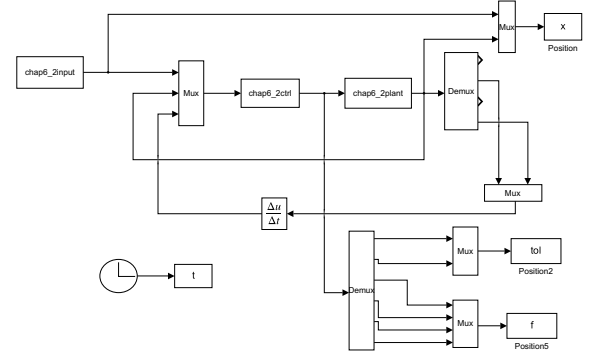


Fig. 12: Simulink implementation of the two-link robotic manipulator control system.

The physical parameters are selected as

$$v = 13.33, \quad q_{01} = 8.98, \quad q_{02} = 8.75, \quad g = 9.8 \text{ m/s}^2.$$

The external disturbance is selected as an error-dependent nonlinear uncertainty:

$$d(t, e, \dot{e}) = d_1 + d_2 \|e\| + d_3 \|\dot{e}\|, \quad (72)$$

where

$$d_1 = [2, 2]^T, \quad d_2 = [3, 3]^T, \quad d_3 = [6, 6]^T.$$

To evaluate the robustness of the proposed controller against modeling uncertainties, a 20% parameter mismatch is introduced into the nominal dynamic model:

$$\Delta M = 0.2M_0, \quad \Delta C = 0.2C_0, \quad \Delta G = 0.2G_0.$$

The desired trajectories of the two joints are selected as

$$q_{1d}(t) = 1 + 0.2 \sin(0.5\pi t), \quad (73)$$

$$q_{2d}(t) = 1 - 0.2 \cos(0.5\pi t). \quad (74)$$

The initial condition is selected as

$$X(0) = [0.6, 0.3, 0.5, 0.5]^T.$$

The control parameters are chosen as

$$Q = 50I_4, \quad \gamma = 20, \quad \sigma = 0.001.$$

In this example, the RBF neural network compensation controller with the second adaptive law ($S = 3, S_1 = 2$) is adopted. The Gaussian basis functions are initialized with

$$\hat{W}(0) = 0.1I.$$

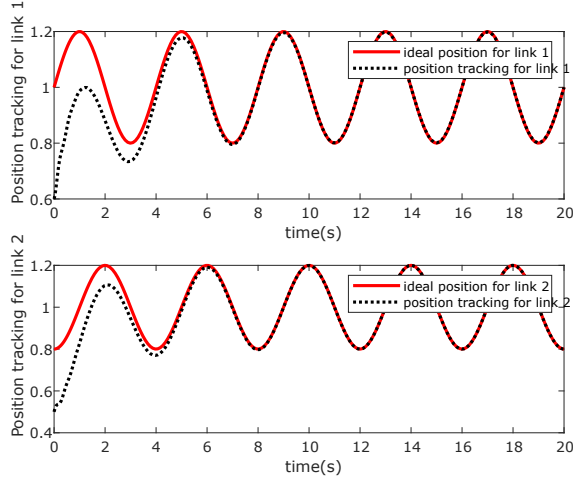


Fig. 13: Angle tracking responses of joint 1 and joint 2 under the proposed RBF adaptive controller ($S = 3, S_1 = 2$).

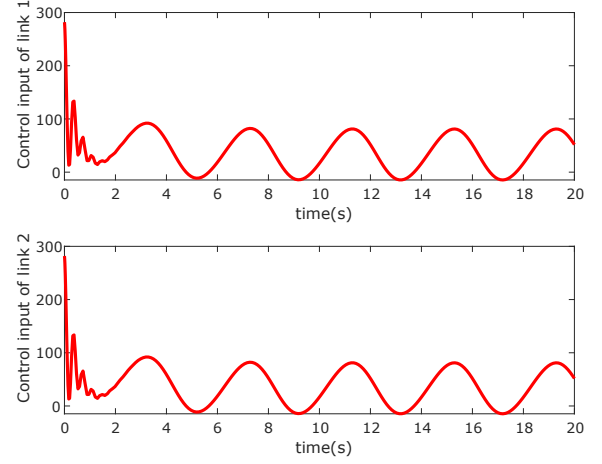


Fig. 15: Control inputs of joint 1 and joint 2 generated by the adaptive controller.

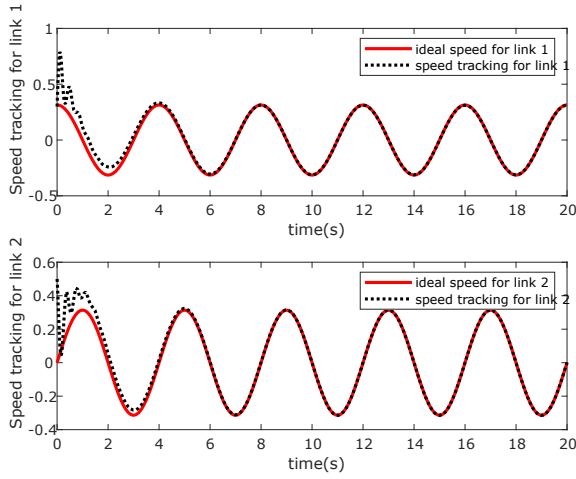


Fig. 14: Angular velocity tracking responses of joint 1 and joint 2.

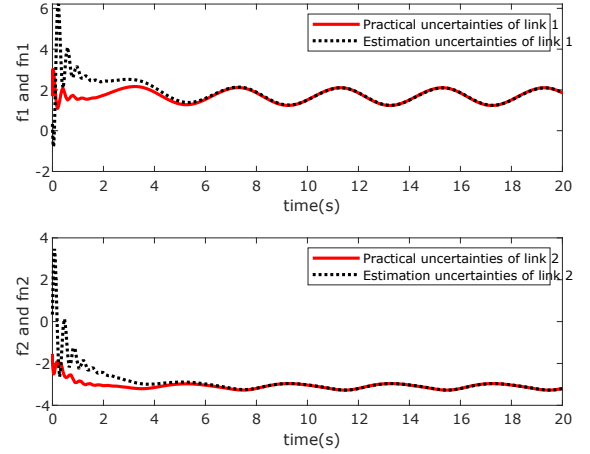


Fig. 16: Actual lumped uncertainty and its RBF neural network approximation.

The RBF network approximates the unknown lumped uncertainty as

$$\hat{f}(x) = \hat{W}^T h(x).$$

The simulation results are presented in the following figures. The joint angle tracking responses are shown in Fig. 13, while the corresponding angular velocity tracking performance is presented in Fig. 14.

The generated control inputs of the two joints are illustrated in Fig. 15.

Finally, the actual lumped uncertainty and its RBF approximation are shown in Fig. 16.

The simulation results demonstrate that the proposed adaptive RBF compensation controller achieves accurate trajectory tracking despite the existence of nonlinear coupling, parameter uncertainties, and external disturbances.

As shown in Fig. 13 and Fig. 14, both joint positions and angular velocities converge to their desired trajectories with

small tracking errors.

Furthermore, Fig. 16 verifies that the RBF neural network can effectively approximate the unknown lumped uncertainty online. The generated control inputs remain bounded and smooth, as illustrated in Fig. 15, demonstrating the robustness of the proposed adaptive compensation strategy.

VII. CONCLUSION

This paper has investigated an adaptive neural network-based control strategy for robotic manipulators subject to modeling uncertainties, external disturbances, and unmodeled nonlinear dynamics. By decomposing the robot dynamics into a nominal model and a lumped uncertainty term, an RBF neural network was introduced to approximate the unknown nonlinear function online. Based on the nominal inverse dynamics and neural compensation, a modular controller structure was developed, where the trajectory tracking objective and uncertainty compensation mechanism are separately designed.

A Lyapunov-based adaptive law was derived to update the neural network weights, and a σ -modification technique was incorporated to improve the robustness against persistent approximation errors and disturbances. The stability analysis demonstrated that the closed-loop system is uniformly ultimately bounded, and all system states and neural network weight estimation errors remain bounded under the proposed adaptive control framework.

Simulation studies were conducted on both a nonlinear servo system and a two-link robotic manipulator. The results of the servo system verified that the RBF neural network can effectively approximate unknown non-smooth disturbances and improve tracking performance compared with the nominal controller without compensation. Furthermore, increasing the number of hidden nodes enhanced the approximation capability of the neural network. The two-link manipulator simulation further demonstrated the effectiveness of the proposed controller in the presence of dynamic coupling, parameter uncertainties, and external disturbances, achieving accurate trajectory tracking while maintaining bounded control inputs.

Future work will focus on experimental validation using a physical robotic platform and further improvements of the adaptive compensation mechanism under more challenging conditions, such as time-varying uncertainties, actuator constraints, and communication limitations in networked robotic systems.

VIII. FAQ

A. *What do the uncertainty terms $f(\cdot)$ and $d(t)$ represent, and what assumptions are required for them?*

In the robot dynamics, $d(t)$ represents the unknown external disturbance torque acting on the manipulator, such as unknown payload variations, environmental contact forces, and external disturbances.

However, the external disturbance is only one part of the total uncertainty considered in this work. Since the exact dynamic model of a practical robot is usually unavailable, the differences between the actual dynamics and the nominal model, together with external disturbances and unmodeled dynamics, are collected into a lumped uncertainty term.

The lumped uncertainty is defined as:

$$f(q, \dot{q}, \ddot{q}) = d - \Delta M(q)\ddot{q} - \Delta C(q, \dot{q})\dot{q} - \Delta G(q)$$

where ΔM , ΔC , and ΔG denote the modeling errors between the actual robot dynamics and the nominal model.

Therefore, $f(\cdot)$ represents the overall unknown nonlinear uncertainty, including modeling uncertainties, parameter variations, external disturbances, and unmodeled effects. The RBF neural network is used to estimate this lumped uncertainty rather than the external disturbance $d(t)$ alone:

$$\hat{f}(x) = \hat{W}^T h(x)$$

where $\hat{f}(x)$ denotes the neural-network estimation of the lumped uncertainty. After expressing the lumped uncertainty as a function of the neural-network input vector x , it can be represented as:

$$f(x) = W^{*T} h(x) + \varepsilon(x)$$

where W^* denotes the ideal weight matrix and $\varepsilon(x)$ denotes the bounded approximation error.

For the stability analysis, the uncertainty terms are assumed to satisfy the following conditions.

1) *Uniform boundedness*: The unknown external disturbance is assumed to be bounded. That is, there exists an unknown positive constant d_m such that:

$$\|d(t)\| \leq d_m, \quad \forall t \geq 0$$

This assumption indicates that the external torque applied to the robot is physically limited. Without bounded disturbances, the adaptive parameters may suffer from parameter drift, and the uniform ultimate boundedness (UUB) of the closed-loop system cannot be guaranteed.

2) *Continuity of the uncertainty*: If the disturbance is only a time-dependent signal $d(t)$, it is generally assumed to be piecewise continuous. This allows finite and bounded sudden changes, such as an abrupt payload change or a short contact impact.

If the uncertainty is state-dependent, i.e., $f = f(x)$ or $d = d(x, t)$, it is commonly assumed to be Lipschitz continuous with respect to the system state over a compact domain Ω_x :

$$\|f(x_1) - f(x_2)\| \leq L_f \|x_1 - x_2\|$$

where L_f is the Lipschitz constant.

This condition guarantees the smoothness of the unknown nonlinear uncertainty and ensures that the RBF neural network can approximate it with a bounded approximation error:

$$f(x) = W^{*T} h(x) + \varepsilon(x)$$

where $\varepsilon(x)$ denotes the bounded approximation error.

3) *Bounded derivative assumption*: Some advanced control methods require the derivative of the disturbance to be bounded:

$$\|\dot{d}(t)\| \leq d_{dm} < \infty.$$

However, this assumption is not necessary for the RBF adaptive controller proposed in this work because the adaptive law does not require the estimation or differentiation of $\dot{d}(t)$.

In summary, the essential assumption in this work is that the lumped uncertainty $f(\cdot)$ is bounded and can be approximated by the RBF neural network with a bounded approximation error. The external disturbance $d(t)$ is considered as one component of this lumped uncertainty and is assumed to be piecewise continuous and uniformly bounded.

B. *How are the RBF centers and widths selected, and how is the curse of dimensionality addressed for high-dimensional robotic systems?*

For a multi-degree-of-freedom robotic system, the input vector of the RBF neural network is defined as the tracking error state:

$$x = X = \begin{bmatrix} e \\ \dot{e} \end{bmatrix} \in \mathbb{R}^{2n},$$

where n denotes the number of robot joints. Therefore, each RBF center is a vector in the same state space:

$$c_i \in \mathbb{R}^{2n}, \quad i = 1, 2, \dots, m.$$

The Gaussian basis function is defined as:

$$h_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2b_i^2}\right),$$

where c_i and b_i represent the center and width of the i -th basis function, respectively.

1) *Selection of RBF centers:* The selection of RBF centers is critical for the approximation capability of the neural network. In this work, the centers are selected using a uniform distribution strategy, which is a commonly adopted and robust approach for low-dimensional control systems. The centers are predefined to cover the expected operating region of the tracking error state space.

It should be noted that each center represents a complete point in the state space:

$$c_i = [c_{i1}, c_{i2}, \dots, c_{i,2n}]^T,$$

rather than a combination of independent nodes assigned to each dimension.

For higher-dimensional robotic systems, uniform placement may become inefficient. A possible improvement is to obtain the centers from collected system trajectories using data-driven methods, such as clustering algorithms (e.g., K-means). This allows the basis functions to concentrate on the regions that are frequently visited during operation, which can be considered as future work.

2) *Selection of RBF widths:* The width parameter b_i determines the coverage range of each Gaussian basis function.

If b_i is too small, each basis function becomes overly localized, resulting in poor generalization capability and slow adaptation. Conversely, if b_i is too large, different basis functions become less distinguishable, which reduces the approximation accuracy.

In this work, a fixed width is adopted for simplicity and computational efficiency:

$$b_i = b, \quad i = 1, 2, \dots, m.$$

An adaptive width selection strategy based on the distance between neighboring centers can further improve the approximation performance, which is also a potential direction for future research.

3) *Avoiding the curse of dimensionality:* A major concern for RBF neural networks in high-dimensional systems is the curse of dimensionality. A traditional grid-based RBF structure assigns multiple nodes independently along each input dimension. If each dimension requires m nodes, the total number of neurons grows exponentially:

$$m^{2n}.$$

Such a structure quickly becomes computationally infeasible as the number of robot joints increases.

However, the RBF structure adopted in this work does not use a grid combination strategy. Instead, a fixed number of local basis functions is directly selected:

$$\{c_1, c_2, \dots, c_m\}.$$

Therefore, the computational complexity of evaluating the hidden layer is approximately:

$$\mathcal{O}(m \cdot 2n),$$

rather than:

$$\mathcal{O}(m^{2n}).$$

This design avoids the exponential growth of neurons with respect to system dimensions. In practical robotic applications, increasing the number of neurons exponentially with the state dimension is unnecessary and computationally impractical. A finite number of properly distributed local basis functions provides a reasonable balance between approximation capability and real-time implementation requirements.

Nevertheless, the approximation accuracy still depends on the number and distribution of selected centers. Therefore, for more complex high-dimensional robotic systems, adaptive center selection or data-driven basis optimization methods can be further investigated.

C. Why is the σ -modification introduced, and how does it affect the tracking error convergence?

The σ -modification is introduced to prevent the parameter drift problem in adaptive control. Without the leakage term, the standard adaptive law is given by

$$\dot{W} = \gamma h(X) X^T P B.$$

This update law behaves as a pure integrator driven by the tracking error state. Although the Lyapunov analysis can guarantee the boundedness of the tracking error, it does not explicitly penalize the magnitude of the estimated weights $\hat{W}$.

In practical systems, the RBF approximation contains an unavoidable bounded error:

$$f(x) = W^{*T} h(x) + \varepsilon(x),$$

where

$$\|\varepsilon(x)\| \leq \varepsilon_0.$$

Therefore, the tracking error generally converges only to a bounded neighborhood of the origin rather than exactly zero. As a result, the adaptation term

$$\gamma h(X) X^T P B$$

may remain persistently excited. If this residual excitation does not vanish, the estimated weights may continue accumulating over time, leading to unbounded parameter estimates. This phenomenon is known as parameter drift.

To suppress parameter drift, the leakage term is introduced:

$$\dot{\hat{W}} = \gamma h(X) X^T P B - \sigma \gamma \|X\| \hat{W},$$

where $\sigma > 0$ is the leakage coefficient.

The additional term

$$-\sigma \gamma \|X\| \hat{W}$$

provides a damping effect on the estimated weights and prevents $\hat{W}$ from growing without bound. However, the introduction of this term also changes the convergence property of the closed-loop system. The asymptotic convergence of the ideal adaptive controller is replaced by uniform ultimate boundedness (UUB).

1) *Ultimate tracking error bound:* According to the Lyapunov analysis, the ultimate bound of the tracking error state can be defined as

$$\rho_X = \frac{2}{\lambda_{\min}(Q)} \left(\frac{\sigma}{4} W_{\max}^2 + \|PB\| \varepsilon_0 \right),$$

where $W_{\max}$ is the upper bound of the ideal neural-network weights, ε_0 denotes the bounded RBF approximation error, and P, Q are determined by the Lyapunov equation

$$A^T P + P A = -Q.$$

Therefore, the closed-loop state satisfies

$$\|X(t)\| \leq \rho_X, \quad t \rightarrow \infty.$$

This expression reveals the trade-off introduced by the σ -modification. A larger leakage coefficient σ provides stronger suppression of parameter drift, but increases the ultimate tracking error bound:

$$\sigma \uparrow \Rightarrow \rho_X \uparrow.$$

Therefore, σ should not be selected excessively large. A suitable value is required to balance weight boundedness and tracking accuracy.

2) *Influence of the control gains K_p and K_v :* The proportional and derivative gains determine the nominal tracking dynamics:

$$\ddot{e} + K_v \dot{e} + K_p e = 0.$$

The control gains affect the ultimate bound indirectly through the closed-loop system matrix:

$$A = \begin{bmatrix} 0 & I \\ -K_p & -K_v \end{bmatrix},$$

which determines the Lyapunov matrices P and Q . Therefore, the relationship between K_p, K_v and the ultimate bound is implicit:

$$K_p, K_v \rightarrow A \rightarrow P, Q \rightarrow \rho_X.$$

A simple explicit expression of $\rho_X = f(K_p, K_v)$ is generally unavailable because P is obtained from the Lyapunov equation.

Although increasing K_p and K_v can improve transient tracking performance and reduce the theoretical error bound, excessively large gains may introduce practical problems.

First, large feedback gains increase the required control torque:

$$\tau \approx M_0(q)(\ddot{q}_r - K_v \dot{e} - K_p e),$$

which may exceed the physical actuator limitation:

$$|\tau| > \tau_{\max}.$$

This results in actuator saturation.

Second, large derivative gains may amplify measurement noise. Since the velocity signal usually contains noise:

$$\dot{q}_m = \dot{q} + n(t),$$

the derivative feedback term becomes

$$-K_v \dot{q}_m = -K_v \dot{q} - K_v n(t).$$

Therefore, excessive K_v may introduce high-frequency control oscillations.

Finally, high feedback gains increase the closed-loop bandwidth and may excite unmodeled high-frequency dynamics, such as flexible joints, friction dynamics, or actuator dynamics.

3) *Practical gain selection:* Therefore, the control gains are not selected by simply minimizing the theoretical ultimate bound. Instead, they should be chosen according to the desired closed-loop dynamics and practical constraints, including actuator limitations, measurement noise, and unmodeled dynamics.

In this work, the RBF neural network is introduced to approximate the unknown nonlinear dynamics:

$$\hat{f}(x) = \hat{W}^T h(x),$$

which reduces the necessity of using excessively large feedback gains. Hence, the controller achieves a balance between tracking performance, robustness, and practical implementability.

D. Does the neural-network weight estimation converge to the ideal weight under the persistency of excitation condition?

The Lyapunov stability analysis in this work proves that both the tracking error state X and the neural-network weight estimation error $\tilde{W}$ are uniformly ultimately bounded. However, boundedness of $\tilde{W}$ does not necessarily imply that the estimated weight matrix converges to the ideal value:

$$\hat{W}(t) \rightarrow W^*.$$

In adaptive control, the convergence of the parameter estimation error requires an additional condition known as the persistency of excitation (PE) condition.

1) *Persistency of excitation condition*: For the RBF neural network,

$$f(x) = W^{*T}h(x) + \varepsilon(x),$$

the weight adaptation is driven by the neural basis vector $h(x)$. The PE condition requires that the regressor vector contains sufficient excitation information over time. Specifically, there exist constants $T > 0$ and $\alpha > 0$ such that

$$\int_t^{t+T} h(\tau)h^T(\tau) d\tau \geq \alpha I, \quad \forall t \geq 0.$$

Physically, this condition means that the robot trajectory must continuously explore different regions of the state space. Under sufficient excitation, the neural network receives enough information to identify the unknown nonlinear dynamics.

However, the PE condition is generally difficult to guarantee in practical robotic systems. For example, when a manipulator performs a static positioning task or maintains a constant posture,

$$q = q_d,$$

the tracking error becomes

$$e = 0, \quad \dot{e} = 0.$$

Since the neural-network input in this work is defined as

$$X = \begin{bmatrix} e \\ \dot{e} \end{bmatrix},$$

the adaptation signal

$$h(X)X^T P B$$

vanishes. Consequently, the weight update stops and the neural network cannot continue improving its estimate.

2) *Effect of insufficient excitation*: Without the PE condition, the adaptive controller does not guarantee parameter convergence:

$$\hat{W} \neq W^*.$$

Instead, the estimated weights may converge to a bounded but non-optimal value. This phenomenon is referred to as the lack of parameter convergence.

It is important to distinguish between stability and learning convergence. The objective of the proposed adaptive controller is not exact identification of the unknown dynamics, but online compensation of the lumped uncertainty:

$$\hat{f}(x) = \hat{W}^T h(x),$$

such that the tracking error remains bounded despite modeling uncertainties and external disturbances.

3) *Relationship with the σ -modification*: The σ -modification introduced in this work is mainly designed to prevent parameter drift rather than to guarantee parameter convergence.

The adaptive law is given by

$$\dot{\hat{W}} = \gamma h(X)X^T P B - \sigma \gamma \|X\| \hat{W}.$$

The leakage term

$$-\sigma \gamma \|X\| \hat{W}$$

provides additional damping on the estimated weights and prevents unbounded parameter growth when persistent disturbances or approximation errors exist.

For a static condition where

$$X = 0,$$

both the learning term and the leakage term vanish:

$$\dot{\hat{W}} = 0.$$

Therefore, the estimated weights remain constant rather than drifting indefinitely. However, without sufficient excitation, the final value of $\hat{W}$ is not guaranteed to be the optimal weight W^* .

4) *Possible methods to improve parameter convergence*: If parameter convergence is required, additional mechanisms are necessary, such as:

- designing sufficiently rich reference trajectories to satisfy the PE condition;
- employing concurrent learning methods that utilize historical data without requiring continuous excitation;
- introducing data-driven approaches such as experience replay for adaptive parameter updating.

These approaches can improve the learning capability of adaptive neural controllers and are considered possible directions for future work.

In summary, the proposed controller guarantees bounded tracking performance and bounded neural-network weights through Lyapunov analysis and the σ -modification. However, exact convergence of the estimated weights to the ideal weights requires additional excitation conditions, which are not assumed in this work.

IX. PYTHON REPRODUCTION RESULTS

To further verify the correctness and reproducibility of the MATLAB simulations, all simulation cases were independently reimplemented in Python. The following figures present the corresponding Python-based simulation results.

It should be noted that when $S = 1$ (computed torque control without compensation) or $S = 2$ (computed torque control with exact feedforward disturbance compensation), the controller does not involve the RBF neural network module. Therefore, the parameters S_1 and m have no effect on these two control modes.

However, the plotting scripts were implemented using a unified parameter naming format. As a result, some figures

corresponding to $S = 1$ and $S = 2$ cases may still contain S_1 and m labels in their filenames or captions. These parameters should be ignored for $S = 1$ and $S = 2$ cases and are only meaningful for the RBF-based compensation case ($S = 3$).

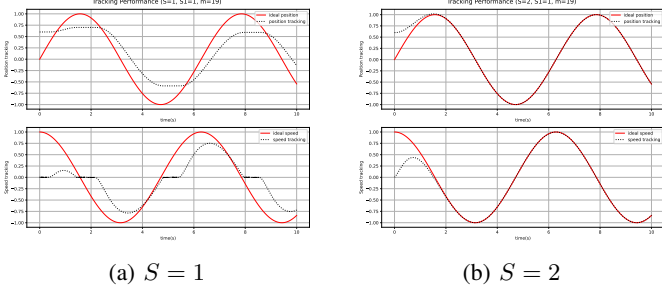


Fig. 17: Simulation1: Python-based tracking results without RBF compensation.

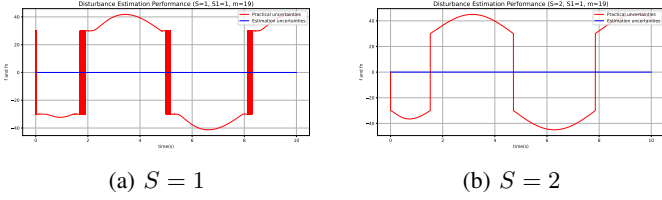


Fig. 18: Simulation1: Python-based disturbance responses for controllers without RBF compensation.

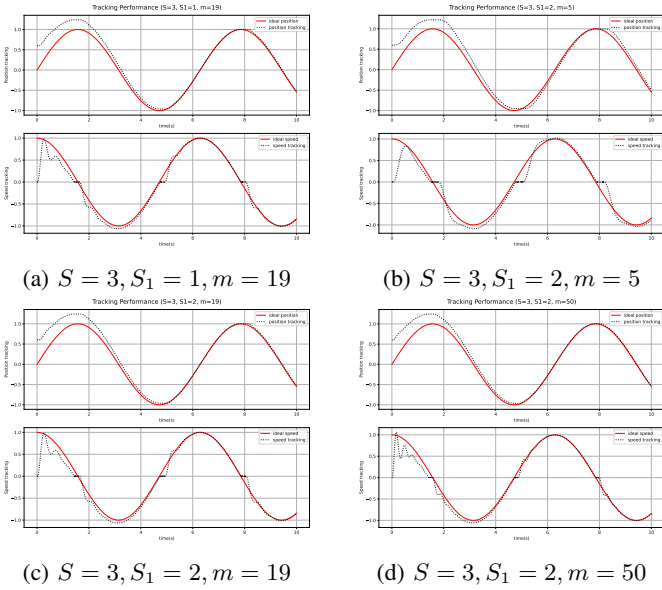


Fig. 19: Simulation1: Python-based tracking results with RBF neural network compensation under different adaptive laws and numbers of hidden nodes.

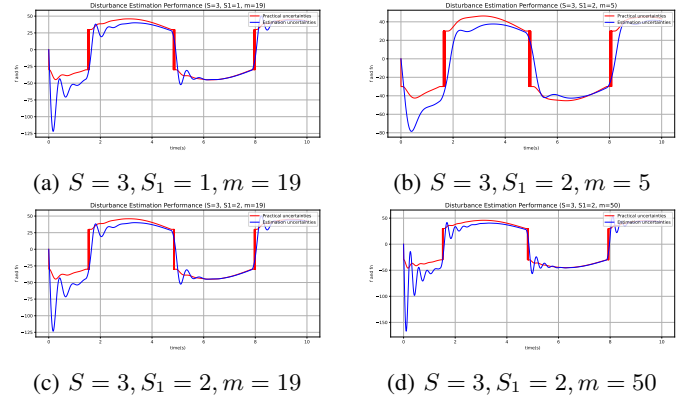


Fig. 20: Simulation1: Python-based disturbance estimation results with RBF compensation.

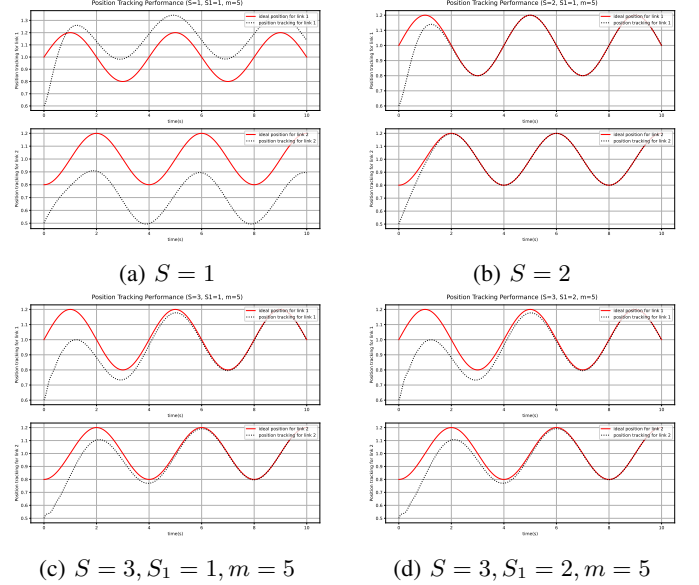


Fig. 21: Simulation2: Python-based position tracking results for the two-link manipulator.

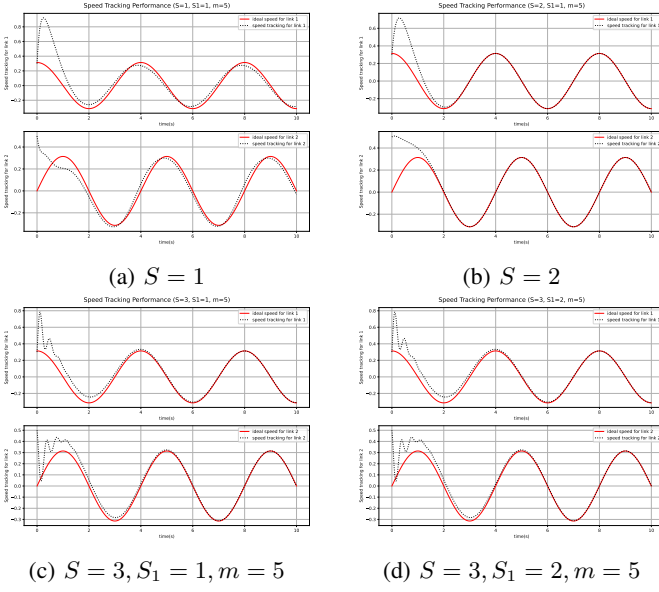


Fig. 22: Simulation2: Python-based velocity tracking results for the two-link manipulator.

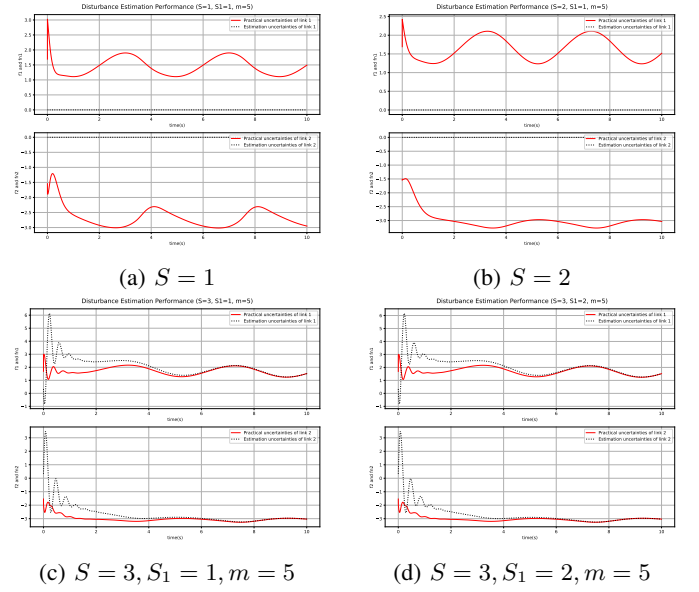


Fig. 24: Simulation2: Python-based disturbance estimation results for the two-link manipulator.

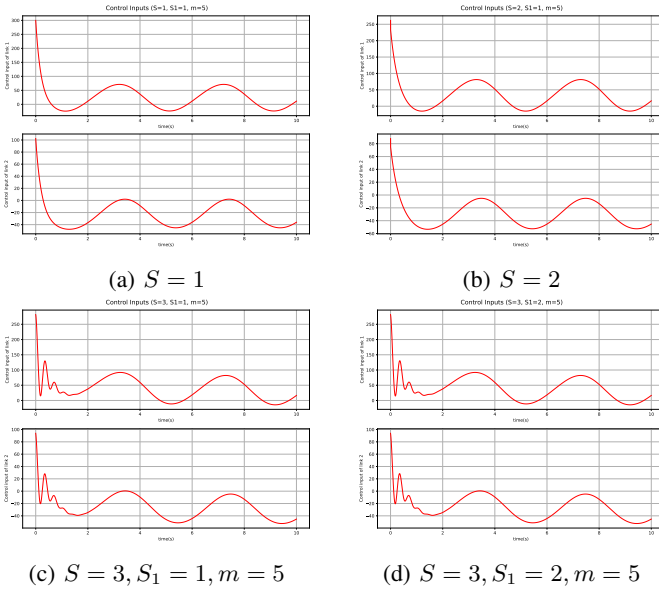


Fig. 23: Simulation2: Python-based control input responses for different controllers.